CS 374: Section A - Homework 6, Problem 1

Anirudh Konidala

October 13, 2026

Throughout, a set S of days off is **valid for (ℓ, k)** if every k consecutive workdays contain at most ℓ days of S. Joy values $J[i]$ may be negative, so an optimal plan may take no days off, with total joy 0.

(a) At most 2 days off in any 28 consecutive workdays

Recursive function

For $1 \le i \le n+1$ and $0 \le g \le 27$, let $P(i,g) = \{i-g\}$ if $g \ge 1$, and $P(i,0) = \varnothing$. Define

$$\boxed{\begin{array}{c} \mathit{MaxJoy}(i,g) = \text{the maximum of } \sum_{s\in S} J[s] \text{ over all sets } S \subseteq \{i, i+1, ..., n\} \\ \text{such that } S \cup P(i,g) \text{ is valid for } (2,28). \end{array}}$$

In words: $\mathit{MaxJoy}(i,g)$ is the most joy we can collect from days i through n, if day $i-g$ is already taken off (or no earlier day matters, when $g=0$). The set $S=\varnothing$ is always allowed, so $\mathit{MaxJoy}(i,g) \ge 0$. We need $\mathit{MaxJoy}(1,0)$.

Why this is the right subproblem. Let H be the days off already chosen before day i, and assume H contains at most one day in $[i-27, i-1]$, namely day $i-g$ (or no day, if $g=0$). Any 28 consecutive days that contain a day $\ge i$ start at day $i-27$ or later, so they meet H only in $P(i,g)$. Hence, if H is valid, $H \cup S$ is valid exactly when $S \cup P(i,g)$ is valid. Every recursive call below keeps this property.

Recurrence

Let $\mathit{next}(g) = g+1$ if $1 \le g \le 26$, and $\mathit{next}(g) = 0$ if $g \in \{0, 27\}$. For all $i > n$ we set $\mathit{MaxJoy}(i,g) = 0$. For $1 \le i \le n$:

$$\mathit{MaxJoy}(i,0) = \max\begin{cases} \mathit{MaxJoy}(i+1,0) & \text{(work day } i\text{)} \\ J[i] + \mathit{MaxJoy}(i+1,1) & \text{(take off day } i\text{)} \end{cases}$$

and for $1 \le g \le 27$:

$$\mathit{MaxJoy}(i,g) = \max\begin{cases} \mathit{MaxJoy}(i+1, \mathit{next}(g)) & \text{(work day } i\text{)} \\ J[i] + \mathit{MaxJoy}(i+28-g, 28-g) & \text{(take off day } i\text{)} \end{cases}$$

Explanation. Every valid S either contains day i or does not, so we take the better of the two options.

- **Work day i.** The next subproblem starts at day $i+1$, whose window of the 27 previous days is $[i-26, i]$. Day $i-g$ stays in that window, now at offset $g+1$, exactly when $1 \le g \le 26$; if $g = 27$ it leaves the window, and if $g = 0$ there is still no day off in the window. This is $next(g)$.
- **Take off day i when $g = 0$.** No earlier day off lies in $[i-27, i-1]$, so day i is the only day off so far in any 28 consecutive days that contain it. Day i is also the only day off in the window $[i-26, i]$ of day $i+1$, at offset 1.
- **Take off day i when $g \ge 1$.** Days $i-g$ and i are both off. The 28 consecutive days $[i-g, i-g+27]$ already contain two days off, so none of the days $i+1, ..., i-g+27$ may be taken off. The first day we may take off next is $i' = i-g+28 \ge i+1$. Among the days $[i'-27, i'-1] = [i-g+1, i-g+27]$ the only day off is day i, at offset $i'-i = 28-g$, which lies in $[1, 27]$. Taking off day i itself is allowed, because the 28 days ending at day i contain only days $i-g$ and i from the plan.

Memoization and evaluation order

We store $MaxJoy$ in a two-dimensional array $MaxJoy[1..n+28][0..27]$. All rows $i > n$ are 0; the largest row the recurrence reads is $i+28-g \le n+27$. Each entry $MaxJoy[i, g]$ depends only on entries in rows larger than i, so we fill rows $i = n, n-1, ..., 1$ in decreasing order, and within a row the columns $g = 0, 1, ..., 27$ in any order.

```
MaxJoyA(J[1..n]):
    for i ← n+1 to n+27 and g ← 0 to 27: MaxJoy[i, g] ← 0
    for i ← n down to 1
        MaxJoy[i, 0] ← max{MaxJoy[i+1, 0], J[i] + MaxJoy[i+1, 1]}
        for g ← 1 to 27
            t ← 28 − g      ⟨⟨offset of day i seen from day i + t⟩⟩
            MaxJoy[i, g] ← max{MaxJoy[i+1, next(g)], J[i] + MaxJoy[i+t, t]}
    return MaxJoy[1, 0]
```

Running time

The array has $28(n+27) = O(n)$ entries, each computed in $O(1)$ time, so the algorithm runs in

$O(n)$ time.

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.

(b) At most ℓ days off in any k consecutive workdays

We assume $\ell \geq 1$; if $\ell = 0$, no day may be taken off and the answer is 0.

Recursive function

The state now records up to $\ell - 1$ recent days off. For $1 \leq i \leq n+1$, let D be any set of at most $\ell - 1$ days in the range $[\max\{1, i-k+1\}, i-1]$. Define

$$\boxed{\begin{array}{c} \mathit{MaxJoy}(i, D) = \text{the maximum of } \sum_{s \in S} J[s] \text{ over all sets } S \subseteq \{i, i+1, ..., n\} \\ \text{such that } S \cup D \text{ is valid for } (\ell, k). \end{array}}$$

In words: $\mathit{MaxJoy}(i, D)$ is the most joy we can collect from days i through n, if the days off among the $k-1$ days before day i are exactly the days in D. Again $S = \varnothing$ is allowed, so $\mathit{MaxJoy}(i, D) \geq 0$. We need $\mathit{MaxJoy}(1, \varnothing)$.

Why this is the right subproblem. As in part (a), let H be a valid set of days off before day i with $H \cap [i-k+1, i-1] = D$. Any k consecutive days that contain a day $\geq i$ start at day $i-k+1$ or later, so they meet H only in D. Hence $H \cup S$ is valid exactly when $S \cup D$ is valid. Part (a) is the special case $\ell = 2$, $k = 28$, $D = P(i, g)$.

Recurrence

For all $i > n$ we set $\mathit{MaxJoy}(i, D) = 0$. For $1 \leq i \leq n$, let $H = D \cup \{i\}$ and define the next day we may take off after day i:

$$i' = \begin{cases} i+1 & \text{if } |H| < \ell \\ \max\{i+1, \min(H) + k\} & \text{if } |H| = \ell \end{cases}$$

Let $D' = H \cap [i'-k+1, i'-1]$. Then

$$\mathit{MaxJoy}(i, D) = \max \begin{cases} \mathit{MaxJoy}(i+1, D \setminus \{i-k+1\}) & \text{(work day } i\text{)} \\ J[i] + \mathit{MaxJoy}(i', D') & \text{(take off day } i\text{)} \end{cases}$$

Explanation.

- **Work day i.** The window of the $k-1$ days before day $i+1$ is $[i-k+2, i]$, so day $i-k+1$ leaves the state if it was in D, and nothing is added.
- **Take off day i.** This is always allowed: any k consecutive days that contain day i meet the earlier plan only inside $[i-k+1, i-1]$, that is, inside D, so they contain at most $|D|+1 \leq \ell$ days off up to day i. If $|H| < \ell$, the next free choice is day $i+1$. If $|H| = \ell$, the k consecutive days starting at day $\min(H)$ already contain the ℓ days of H, so no day before $\min(H) + k$ (and after i) may be taken off; the next possible day is i'. The new state D' keeps the days of H inside the $k-1$ days before i'.

- **D' is a legal state.** If $|H| < \ell$ then $|D'| \le \ell - 1$. If $|H| = \ell$ then $\min(H) \le i' - k$, so $\min(H) \notin D'$ and again $|D'| \le \ell - 1$.

Every valid S either contains day i or does not, and in each case the days of S before i' are forced, so the maximum of the two options is correct.

Memoization and evaluation order

Because ℓ is a constant, we encode D by its offsets $i - d$ for $d \in D$. Each offset lies in $[1, k-1]$. We sort the offsets in increasing order and pad them with zeros to a tuple $(o_1, ..., o_{\ell-1}) \in \{0, ..., k-1\}^{\ell-1}$. We store *MaxJoy* in an ℓ-dimensional array $MaxJoy[1..n+k][0..k-1]\cdots[0..k-1]$, with one index for the day i and $\ell - 1$ indices for the offset tuple. All rows $i > n$ are 0; the largest day the recurrence reads is $\min(H) + k \le n + k - 1$.

Every entry for day i depends only on entries for larger days ($i + 1$ or $i' > i$). So the outer loop runs over $i = n, n-1, ..., 1$ in decreasing order, and the inner loop runs over every sorted, zero-padded offset tuple in lexicographic order (any order works inside one day). The answer is the entry for $i = 1$ and the all-zero tuple, which encodes $D = \varnothing$.

Running time

For each day i there are at most $k^{\ell-1}$ offset tuples, so the array has $O\big((n+k)k^{\ell-1}\big) = O\big(nk^{\ell-1}\big)$ entries, using $k \le n$. Each entry is computed in $O(\ell)$ time: we form H, find $\min(H)$, compute i', and re-encode D' and $D \setminus \{i - k + 1\}$ as offset tuples, all on sets of at most ℓ elements. Since ℓ is a constant, the algorithm runs in

$$\boxed{O\big(nk^{\ell-1}\big) \text{ time.}}$$

For $\ell = 2$ and $k = 28$ this is $O(n)$, matching part (a).

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.